

MASTERCLASS

Diseña, prototipa, lanza:

¿El nuevo flujo UX asistido por IA?

De un diseño aprobado en Figma a un prototipo real, deployado y con URL pública — en una sola sesión.

Figma



Claude Code



Next.js



GitHub



Vercel



Supabase



Francisco Hormazábal

UX Engineer & Product Design Consultant

fhormazabal@cleveritgroup.com

Casos reales · Collective Health | Para perfiles UX y Product Designers

fcophox.com

Research & Optimization of digital experiences

Strategy, aesthetics, and usability combined to optimize digital experiences that connect people with the digital world.



Francisco Hormazábal
UX Engineer & Product Design Consultant
fhormazabal@cleveritgroup.com

Experience backed by knowledge

Impacto profesional

- Estrategia UX en múltiples industrias
- Integración de IA en procesos UX
- Optimización de conversión (CRO)
- Investigación en Experiencia de usuario
- Liderazgo UX (11+ diseñadores)
- Desarrollo end-to-end de productos digitales
- Arquitectura de Información y Usabilidad

Tú diriges. La IA ejecuta.

Vibe coding no es “que la IA diseñe por ti”.

Es asumir el rol de director: tú pones el contexto, las reglas y el criterio; la IA pone el código.

La calidad del resultado depende de la calidad de tu dirección, la organización y el orden.

TÚ (director)

Contexto, brief, design system, criterio
UX, validación

LA IA (ejecutante)

Código, componentes, estados,
refactors, comandos

El resultado

Prototipo navegable, versionado y
deployado

¿POR QUÉ ESTA CLASE?

Lo que esperamos para hoy

Hasta hace poco, el flujo era así: entregabas un Figma impecable... y esperabas **días o semanas** para verlo funcionando.

El feedback llegaba tarde y las decisiones ya estaban tomadas.

50 min

de brief a URL pública

2 casos

reales de un cliente

0 líneas

escritas a mano

HOY, CON IA BIEN USADA

Puedes ser tú quien lo vea funcionando en producción esa misma tarde.

No para reemplazar a tu dev — sino para llegar al handoff con algo mucho más concreto que un click-through.

¿Qué tipo de diseñador quieres ser?

1

Diseñador tradicional

Entrega Figma y espera. El prototipo vive solo en el mockup.

2

Diseñador que prototipa

Arma click-throughs navegables. Valida flujo, pero no producto.

3

Diseñador que produce

Entrega prototipos reales, deployados, con URL pública. El destino de hoy.

META DE HOY

Al salir de esta sesión vas a poder...

- 1** **claude.md** Explicar y escribir el brief persistente de tu proyecto para cualquier IA.
- 2** **design.md** Construir un master control de estilos: tokens, tipografías y componentes permitidos.
- 3** **Vibe coding** Generar componentes Next.js + Tailwind con prompts estructurados en Claude Code.
- 4** **Git + Deploy** Versionar en GitHub y publicar en Vercel sin salir de la terminal.
- 5** **Iterar desde el código** Cambiar un token en design.md y propagarlo a toda la app.

AGENDA

50 minutos, 3 bloques

TEORÍA

20 min

- Panorama del UX Workflow con IA
- claude.md: el brief de tu proyecto
- design.md: design system vivo
- Vibe coding, stack y flujo terminal

PRÁCTICA

30 min

- Setup: Next.js + Tailwind + Claude Code
- Caso 1 · Límite de fertilidad (guiado)
- Caso 2 · Verificación de TIN (en parejas)
- Deploy a Vercel con URL pública

CIERRE

2 min

- 3 takeaways
- Pregunta final
- Entregables en PDF



Principio de oro: si el tiempo aprieta, recorta el live-coding y muestra el resultado hecho. Lo importante es ver el flujo completo.

MÓDULO 1 · 20 MIN

Teoría: las piezas del flujo

Cuatro ideas simples que cambian cómo trabajas: el workflow con IA, claude.md, design.md y vibe coding.



Las 5 etapas del flujo asistido



✶ Ojo: son las mismas etapas del UX de siempre. Lo que cambia es la velocidad con que las recorres.

Reflexión

“El flujo no cambia las etapas del UX,
las acelera.”

Lo que antes tomaba una semana de investigación + ideación + prototipado + handoff, hoy se comprime en una sesión — si sabes qué pedirle a la IA.

claude.md · el brief de tu proyecto

Un archivo de texto plano en la raíz del proyecto que la IA lee automáticamente en cada sesión.

Piénsalo como el brief que le pasarías a un diseñador nuevo el día 1:

- contexto
- stack
- reglas
- restricciones
- ejemplos de cómo se hacen las cosas acá.

“Es como tener un PM, un tech lead y un design system manager sentados contigo en cada sesión.”

```
mi-prototipo/  
├─ app/  
├─ claude.md ← el brief  
├─ design.md  
└─ package.json
```

Las 6 secciones de un buen `claude.md`

1 Contexto del proyecto

Qué producto es, quién lo usa, qué problema resuelve.

2 Stack técnico

Framework, estilos, componentes, versiones.

3 Convenciones de código

Naming, estructura de carpetas, cómo se nombra todo.

4 Reglas de negocio

Qué se puede y qué no. Validaciones y casos borde.

5 Definición de “Done”

Qué significa que una pantalla está terminada.

6 Ejemplos

Links a componentes existentes que sirven de referencia.

✳ Recuerda: Existe un material en PDF “Banco de prompts + templates” donde va la plantilla completa lista para copiar.

design.md · tu design system, pero vivo

FIGMA (estático)

- Excelente para explorar y comunicar
- Los tokens viven en el archivo de diseño
- Se desconectan del código apenas un dev los implementa a mano
- Actualizar = perseguir pantallas una a una

DESIGN.MD (vivo)

- Tokens, tipografías, spacing y componentes permitidos en un archivo
- La IA lo lee y genera código consistente
- Cambias un color → Claude Code actualiza todos los componentes
- Una sola fuente de verdad, versionada en git

✱ No compiten: Figma para explorar, design.md para que la IA respete y los tokens se propaguen solos.

¿Qué vive dentro de design.md?

Color tokens	Primarios, neutros, semánticos, estados	--color-primary: #3B82F6
Tipografía	Familia, tamaños, pesos, line-height	--font-size-base: 16px
Spacing	Escala base 4px y múltiplos	--space-2: 8px
Radius	Border radius por tamaño	--radius-md: 6px
Componentes	Lista blanca de lo permitido	Button, Input, Card, Badge
Reglas de uso	Cuándo usar cada componente y estado	“Un solo CTA primario por pantalla”

✱ Comenta que, si la IA propone un componente fuera de la lista, debe pedir permiso primero.

¿Qué es vibe coding (bien hecho)?

Programar en colaboración con la IA desde la terminal.

Tú das el contexto, los objetivos y las restricciones. La IA propone código, hace ajustes, ejecuta comandos y valida resultados.

Tú eres el director. Ella, la ejecutante técnica.

MAL VIBE CODING

“Hazme una app bonita” y aceptar lo primero que salga, sin contexto ni criterio.

BUEN VIBE CODING

claude.md + design.md + prompts estructurados + validación en cada paso.



Se que no te gusta, pero es un concepto usado hoy.

El stack de esta masterclass

Next.js 14+

Framework base (App Router). Lo que el dev ya usa.

Tailwind CSS

Estilos utility-first. Consume los tokens de design.md.

Claude Code CLI

La herramienta de vibe coding desde la terminal.

Git + GitHub

Versionado y respaldo, todo desde la terminal.

Vercel

Deploy en 30 segundos: repo conectado, URL pública.

Supabase

Base de datos y auth cuando el prototipo pide datos reales.



Como capa opcional pero recomendada: shadcn/ui — componentes copia-y-pegar que se alinean naturalmente con design.md.

De cero a deploy en 5 comandos

```
npx create-next-app@latest mi-prototipo
# 1. Crear proyecto
cd mi-prototipo
# 2. Entrar a la carpeta
claude
# 3. Abrir Claude Code... y a iterar con prompts
git init && git add . && git commit -m "feat: v1"
# 4. Versionar
npx vercel
# 5. Publicar → URL pública en ~30 segundos
```

LA IDEA

Nunca sales de la terminal.

Crear, iterar, versionar y publicar es un solo flujo continuo.

Eso es todo lo que necesitas de teoría.

MÓDULO 2 · 30 MIN

Práctica: casos reales

Casos del equipo de Collective Health, resueltos en vivo: fertilidad por ciclos y verificación de TIN.



Setup del proyecto

```
npx create-next-app@latest fertility-tin-prototype \
  --typescript --tailwind --app
cd fertility-tin-prototype
touch claude.md design.md # los 2 archivos mágicos
code . # pegar templates de los PDFs
claude # abrir Claude Code
```

CHECKLIST ANTES DE SEGUIR

- ✓ Next.js corre en localhost:3000
- ✓ Claude Code responde al primer prompt
- ✓ claude.md y design.md en la raíz

Límite de fertilidad por ciclos

EL BRIEF DEL CLIENTE

Un Plan Operations Associate necesita configurar la cobertura de fertilidad con un máximo de 3 ciclos de por vida. Hoy solo existen límites por monto y por cantidad. Hay que agregar “ciclo” como nuevo tipo, manteniendo coherencia con el patrón existente.

PATRÓN EXISTENTE

- 1. Seleccionar límite
- 2. Tipo de límite
- 3. Aplicar límite a...

1 Comprender
2 min

2 Generar componente
5 min

3 Microcopy y estados
3 min

4 Commit
2 min

Primero entender, después construir

Lee claude.md y design.md. Eres un Product Designer en Collective Health. El brief:

“agregar límites por ciclo de fertilidad, máx. 3 de por vida, siguiendo el patrón Seleccionar límite → Tipo de límite → Aplicar límite a”.

Dame:

1. Resumen del problema en 3 bullets.
2. Pantallas/componentes que hay que tocar.
3. Riesgos UX que detectas.

POR QUÉ PARTE ASÍ

El primer prompt nunca pide código.

Pide comprensión: obliga a la IA a leer tu contexto antes de proponer nada.

Así dirige un director.

El prompt que genera la pantalla

Crea `<LimitConfigurationFlow />` en `app/components/`:

- 3 pasos: Seleccionar límite / Tipo de límite / Aplicar límite a.
- Step 2: tarjetas "Monto", "Cantidad" y "Ciclo" (esta última con badge NUEVO).
- Step 3 con "Ciclo": input numérico, label "Máximo de ciclos de por vida", helper "Recomendado: 3".
- Usa SOLO componentes y tokens de design.md.
- Estado con useState, sin librerías externas.

FÍJATE EN...

Restricciones explícitas: qué tarjetas, qué labels, qué helper text.

Y dos reglas duras: solo tokens de design.md y sin librerías nuevas.

Una pantalla sin estados no está terminada

ÉXITO

Toast verde: "Límite de ciclos guardado para cobertura de fertilidad".

ERROR

Si ingresa 0 o negativo: "El máximo de ciclos debe ser mayor a 0".

CTA CONTEXTUAL

El botón cambia según el tipo: "Guardar límite de ciclos", "...de monto".



Regla de microcopy: tono constructivo, nunca culpar al usuario. "El TIN no coincide" en vez de "Ingresaste un TIN inválido".

Guardar el avance con un commit claro

```
git init
git add .
git commit -m "feat(caso-1): límite por ciclos"
git branch -M main
```

ENTREGABLE CASO 1

Pantalla funcional con 3 pasos, estados de éxito y error, microcopy contextual y commit en git.

“En 15 minutos pasamos de un brief en lenguaje natural a una pantalla navegable que respeta el patrón existente.”



Esto es lo que cambia cuando el design system vive en el código.”

Verificación de TIN de proveedores

EL BRIEF DEL CLIENTE

Provider Operations necesita buscar un proveedor por TIN o nombre, ver su estado de validación y poder marcar manualmente un TIN como verificado. El proceso actual es manual y poco visible.

DINÁMICA

Prompts iniciales guiados y luego 8 minutos de trabajo en parejas. El instructor circula.

LOS 4 ESTADOS DE UN TIN

Pass

Fail

Manual override

Vendor-check-pending

✳ Cada estado tiene su badge de color — definidos como tokens semánticos en design.md, no inventados por pantalla.

Búsqueda + tabla con 4 estados

Lee `claude.md` y `design.md`. Crea `app/providers/page.tsx`:

1. Barra de búsqueda por TIN o nombre.
2. Tabla: Nombre / TIN / Estado / Acción.
3. Badges de color por estado:
Pass (verde) · Fail (rojo) ·
Manual override (amarillo) · Pending (gris).
4. Mock data: 6 proveedores, al menos uno de cada estado.

TIP DE DIRECCIÓN

Pide mock data con todos los estados representados: así validas la UI completa sin backend.

La acción crítica: marcar como verificado

Agrega a la tabla:

1. Botón "Marcar como verificado" en filas Fail o Vendor-check-pending.
2. Modal de confirmación: "¿Confirmas que el TIN {TIN} de {Nombre} está validado? Esta acción quedará registrada con tu usuario."
3. Al confirmar → estado "Manual override" (amarillo) + toast de éxito.

UX DE CONFIANZA

Acción irreversible = modal de confirmación + registro de auditoría + feedback visible.

Cancelar a la izquierda, confirmar a la derecha.

GitHub + Vercel: la URL pública

```
# 1. Crear repo en GitHub (requiere gh CLI)
gh repo create fertility-tin-prototype \
  --public --source=. --remote=origin
git push -u origin main
```

```
# 2. Deploy a Vercel (auto-detecta Next.js)
npx vercel
# → URL pública en ~30 segundos
```

DESDE AHORA

Con el repo conectado en vercel.com, cada git push a main hace deploy automático.

“Esto no es un Figma clickeable. Es un prototipo real, con URL pública, que puedes mandarle a tu PM, tu stakeholder o tu dev.

EL SIGUIENTE NIVEL

¿Y cuándo entra Supabase?

Cuando el prototipo deja de ser demo y necesita datos reales. Supabase te da base de datos Postgres, auth y APIs sin escribir backend.

Nivel 1 · Mock data

Datos de mentira en el código. Suficiente para validar UI y flujo. Lo que hicimos hoy.

Nivel 2 · Supabase

Los overrides se guardan de verdad: tabla providers, registro de auditoría, login real.

Nivel 3 · Producción

Tu dev toma el repo, endurece la lógica y el prototipo se convierte en la base del producto.

```
npx supabase init → "Conecta este proyecto a Supabase y guarda los overrides en una tabla providers"
(prompt a Claude Code)
```

CIERRE

3 ideas para llevarse hoy

1

claude.md es tu brief persistente

Si la IA no respeta tu contexto, no es porque sea tonta: es porque no se lo diste. Todo proyecto parte por este archivo.

2

design.md es tu design system vivo

No compite con Figma, lo complementa. Figma para explorar; design.md para que la IA respete y los tokens se propaguen solos.

3

El flujo completo toma 50 minutos, no 5 días

Brief → claude.md + design.md → vibe coding → git → Vercel. Repetir esa cadena te convierte en un designer que entrega prototipos reales.

Entonces: “Si mañana te llega un brief nuevo, ¿cuál sería el **primer archivo** que crearías antes de abrir Figma?”

RECURSOS

- Github: <https://docs.github.com/es>
- Claude Code: docs.anthropic.com/claude-code
- Vercel: vercel.com/docs · Supabase: supabase.com/docs
- Libro: “Refactoring UI” (Wathan & Schoger)

ENTREGABLES EN PDF (SE VAN CON USTEDES)

1. Cheat sheet de terminal para perfiles UX
2. Guía básica de Git + GitHub desde terminal
3. Banco de prompts + templates [claude.md](#) / [design.md](#)
4. Guía de deploy: Supabase + Vercel paso a paso

¡Gracias! Ahora, a construir.

Tú diriges. La IA ejecuta. Das a valor a tu equipo y al producto.

Diseña, Prototipa, Lanza · MasterClass UX Workflow con IA · Casos Collective Health



Francisco Hormazábal

UX Engineer & Product Design Consultant

fhormazabal@cleveritgroup.com

Casos reales · Collective Health | Para perfiles UX y Product Designers